Predicting Medical Insurance Charges
Case Study Report

R. Promkam

February 28, 2026

Abstract

We analyse a medical insurance dataset containing 2,772 records with seven demographic and health-related features. The task is a **regression** problem: predicting individual medical insurance charges in US dollars. After removing 1,435 duplicate rows and engineering a smoker–BMI interaction feature, we train and tune seven regression models via grid-search cross-validation. The best-performing model is a random forest regressor, achieving a test R^2 of 0.9012 and a test MAE of \$2,413.46. The results confirm that smoking status, combined with body mass index, is the dominant driver of insurance costs, suggesting that targeted health interventions for smokers with high BMI could yield the largest reduction in charges.

Contents

1 Introduction **2**

2 Data Cleaning **3**
2.1 Column information . . . 4
2.2 Descriptive statistics . . . 4
2.3 Missing values . . . 5
2.4 Duplicate removal . . . 5

3 Exploratory Data Analysis **5**
3.1 Target distribution . . . 5
3.2 Numerical feature distributions . . . 6
3.3 Correlation heatmap . . . 8
3.4 Charges by categorical features . . . 9
3.5 Age vs charges by smoker status . . . 11
3.6 BMI vs charges by smoker status . . . 12

4 Feature Engineering and Data Splitting **13**
4.1 Interaction term and encoding . . . 13
4.2 Train/test split . . . 14
4.3 Feature scaling . . . 14

5 Model Training **15**
5.1 Linear regression . . . 15
5.2 Ridge regression . . . 15
5.3 Lasso regression . . . 16
5.4 Support vector regression . . . 16
5.5 Decision tree . . . 17
5.6 Random forest . . . 17
5.7 Gradient boosting . . . 18

6 Model Evaluation **18**
6.1 Linear regression . . . 18
6.2 Ridge regression . . . 19
6.3 Lasso regression . . . 19
6.4 Support vector regression . . . 19
6.5 Decision tree . . . 19
6.6 Random forest . . . 19
6.7 Gradient boosting . . . 20
6.8 Summary of test results . . . 20

7 Model Comparison **20**
7.1 CV R^2 vs test R^2 . . . 21
7.2 Test MAE comparison . . . 22
7.3 Best model selection . . . 22

8 Model Deployment **23**
8.1 Saving the artifact . . . 23
8.2 Loading and inspecting the artifact . . . 23
8.3 Prediction on a new observation . . . 24

9 Conclusion **25**

1 Introduction

Medical insurance pricing relies on actuarial models that estimate expected costs from policyholder characteristics such as age, sex, body mass index (BMI), number of dependants, smoking status, and geographic region. Accurate charge prediction benefits both insurers, who must set sustainable premiums, and policyholders, who deserve transparent pricing.

Accurately predicting medical insurance costs is of paramount importance to both the healthcare and financial sectors. It serves as the foundation for an insurance company's solvency; by quantifying risk, insurers can set premiums that adequately cover expected claims while maintaining necessary reserves for catastrophic events [2]. The shift toward machine learning models, as opposed to traditional actuarial tables, allows for dynamic, data-driven pricing that better captures complex, non-linear relationships in health data, thereby informing more personalized and equitable insurance plans.

In this case study we formulate the problem as a supervised regression task: given a feature vector describing a policyholder, we predict the corresponding annual medical insurance charge. We pursue the following objectives:

(i) Clean the raw dataset and perform exploratory data analysis to identify key predictors.

(ii) Engineer informative features, in particular a smoker–BMI interaction term.

(iii) Train seven regression models with hyperparameter tuning via five-fold cross-validation.

(iv) Evaluate all models on a held-out test set and select the best performer.

(v) Deploy the chosen model as a serialised artifact for future inference.

The remainder of this report is organised as follows. Section 2 describes data cleaning. Section 3 presents the exploratory data analysis. Section 4 covers feature engineering and the train/test split. Sections 5 and 6 detail model training and evaluation, respectively. Section 7 compares all models. Section 8 demonstrates model deployment. Section 9 concludes with practical implications and future directions.

2 Data Cleaning

We begin by importing the required libraries and loading the dataset [3]. The imported libraries serve specific roles in our pipeline:

- `numpy` [4] and `pandas` [7] provide core data structures for manipulation and numerical operations.
- `matplotlib` [5] and `seaborn` [9] are used for visualising data distributions and relationships.
- `sklearn` (Scikit-learn [6]) supplies the machine learning algorithms, preprocessing utilities like `StandardScaler`, evaluation metrics such as `mean_absolute_error` and `r2_score`, and model selection tools like `GridSearchCV`.
- `os`, `warnings`, and `pickle` [8] handle file system operations, suppress routine warnings, and allow us to serialise our final model to disk, respectively.

```python
import os
import warnings
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import mean_absolute_error, r2_score

np.random.seed(42)
plt.rcParams.update({'figure.dpi': 120, 'font.size': 11})
os.makedirs('figures', exist_ok=True)
warnings.filterwarnings('ignore')
```

```python
df = pd.read_csv('medical_insurance.csv')
print(f"Shape: {df.shape}")
df.head()
```

The DataFrame `df` has shape $(2772, 7)$, comprising 2,772 observations and seven columns. The first five rows are shown below.

age	sex	bmi	children	smoker	region	charges
19	female	27.900	0	yes	southwest	16,884.92
18	male	33.770	1	no	southeast	1,725.55
28	male	33.000	3	no	southeast	4,449.46
33	male	22.705	0	no	northwest	21,984.47
32	male	28.880	0	no	northwest	3,866.86

The seven columns are as follows:

- `age` (int64) — policyholder age in years.
- `sex` (object) — biological sex, either `female` or `male`.

- `bmi` (float64) — body mass index, a measure of body fat based on height and weight.
- `children` (int64) — number of dependants covered by the insurance plan.
- `smoker` (object) — whether the policyholder smokes (`yes` or `no`).
- `region` (object) — residential region in the US (`southwest`, `southeast`, `northwest`, `northeast`).
- `charges` (float64) — annual medical insurance charges in USD (the target variable).

2.1 Column information

We inspect the column data types and non-null counts to verify structural integrity before proceeding with any transformations.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2772 entries, 0 to 2771
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       2772 non-null   int64
 1   sex       2772 non-null   object
 2   bmi       2772 non-null   float64
 3   children  2772 non-null   int64
 4   smoker    2772 non-null   object
 5   region    2772 non-null   object
 6   charges   2772 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 151.7+ KB
```

All 2,772 entries are non-null across every column. The dataset contains two floating-point columns (`bmi`, `charges`), two integer columns (`age`, `children`), and three categorical object columns (`sex`, `smoker`, `region`).

2.2 Descriptive statistics

We compute summary statistics for all numerical columns to understand the central tendency, spread, and range of each feature.

```
df.describe()
```

	age	bmi	children	charges
count	2,772	2,772	2,772	2,772
mean	39.11	30.70	1.10	13,261.37
std	14.08	6.13	1.21	12,151.77
min	18.00	15.96	0	1,121.87
25%	26.00	26.22	0	4,687.80
50%	39.00	30.45	1	9,333.01
75%	51.00	34.77	2	16,577.78
max	64.00	53.13	5	63,770.43

The mean age is approximately 39 years, ranging from 18 to 64. BMI centres around 30.7, placing the average policyholder in the "obese" category according to the WHO classification. The target variable `charges` has a large standard deviation ($12,151.77) relative to its mean ($13,261.37), indicating substantial variability in insurance costs.

2.3 Missing values

We check for missing data across all columns to determine whether imputation is necessary before modelling.

```
print("Missing values per column:")
print(df.isnull().sum())
print(f"\nTotal missing: {df.isnull().sum().sum()}")
```

Column	Missing
age	0
sex	0
bmi	0
children	0
smoker	0
region	0
charges	0
Total	0

No missing values are present in any column. We proceed directly to duplicate detection.

2.4 Duplicate removal

We detect and remove exact duplicate rows to prevent data leakage and ensure that the training set contains only unique observations.

```
n_dup = df.duplicated().sum()
print(f"Shape before cleaning: {df.shape}")
print(f"Duplicate rows found:  {n_dup}")

if n_dup > 0:
    print("\nSample duplicates:")
    display(df[df.duplicated(keep='first')].head())

df = df.drop_duplicates()
print(f"\nShape after cleaning:  {df.shape}")
```

The variable `n_dup` counts the number of exact duplicate rows. We find $1,435$ duplicates—more than half the dataset. The shape before cleaning is $(2772, 7)$ and after cleaning is $(1337, 7)$, retaining $1,337$ unique records. This large number of duplicates likely arises from the dataset being a concatenation of overlapping sources.

3 Exploratory Data Analysis

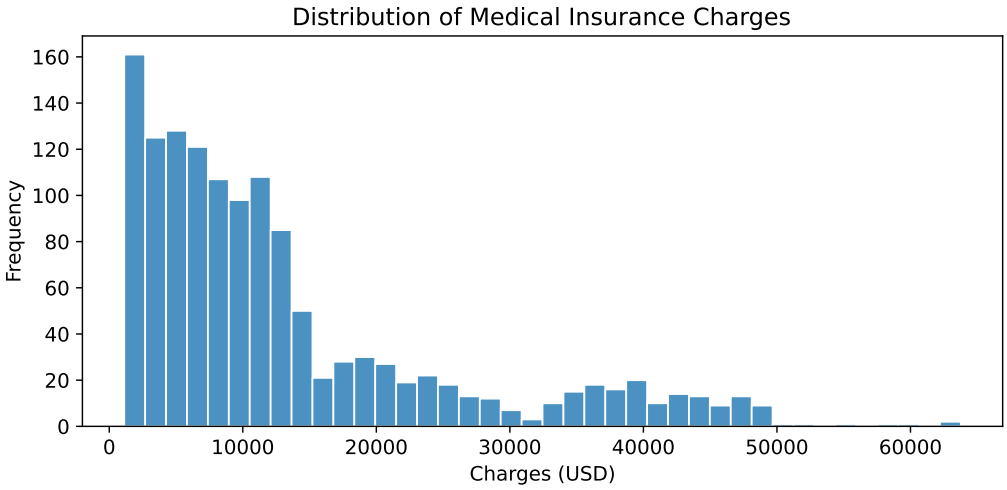
We examine the target distribution, individual feature distributions, correlations among numerical variables, and the relationship between categorical features and charges.

3.1 Target distribution

We begin by plotting the histogram of the target variable to assess its shape, central tendency, and skewness.

```python
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(df['charges'], bins=40, edgecolor='white', alpha=0.8)
ax.set_xlabel('Charges (USD)')
ax.set_ylabel('Frequency')
ax.set_title('Distribution of Medical Insurance Charges')
plt.tight_layout()
fig.savefig('figures/fig_01_target_distribution.pdf', bbox_inches='tight')
plt.show()

print(f"Mean:     ${df['charges'].mean():,.2f}")
print(f"Median:   ${df['charges'].median():,.2f}")
print(f"Skewness: {df['charges'].skew():.2f}")
```

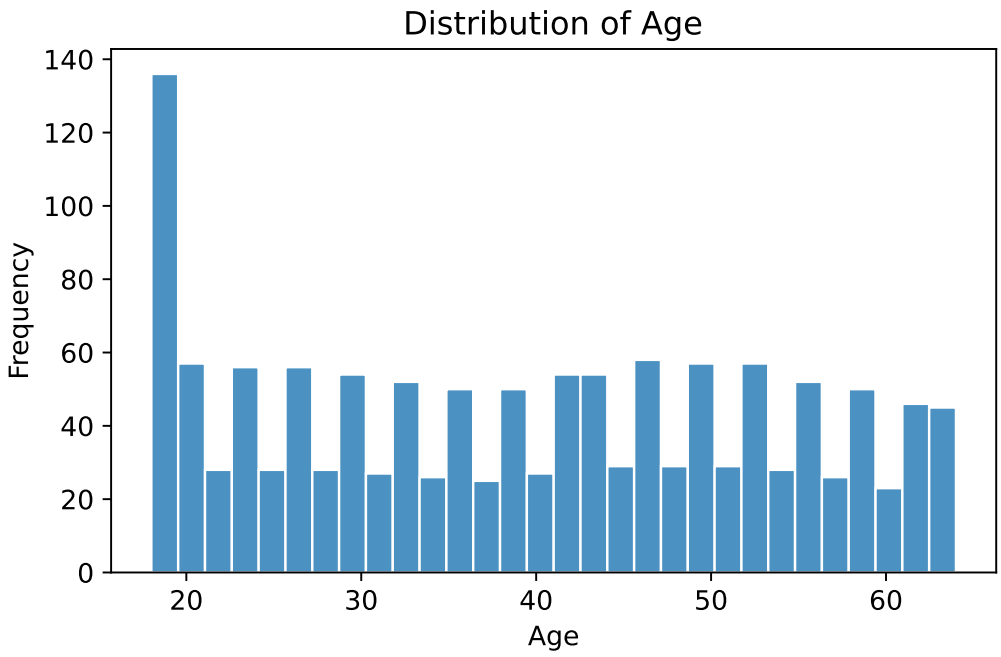


The mean charge is \$13,279.12 while the median is \$9,386.16, confirming the right skew visible in the histogram above. The skewness coefficient is 1.52, indicating a moderately heavy right tail. A small proportion of policyholders incur very high costs (up to \$63,770.43), pulling the mean well above the median.

3.2 Numerical feature distributions

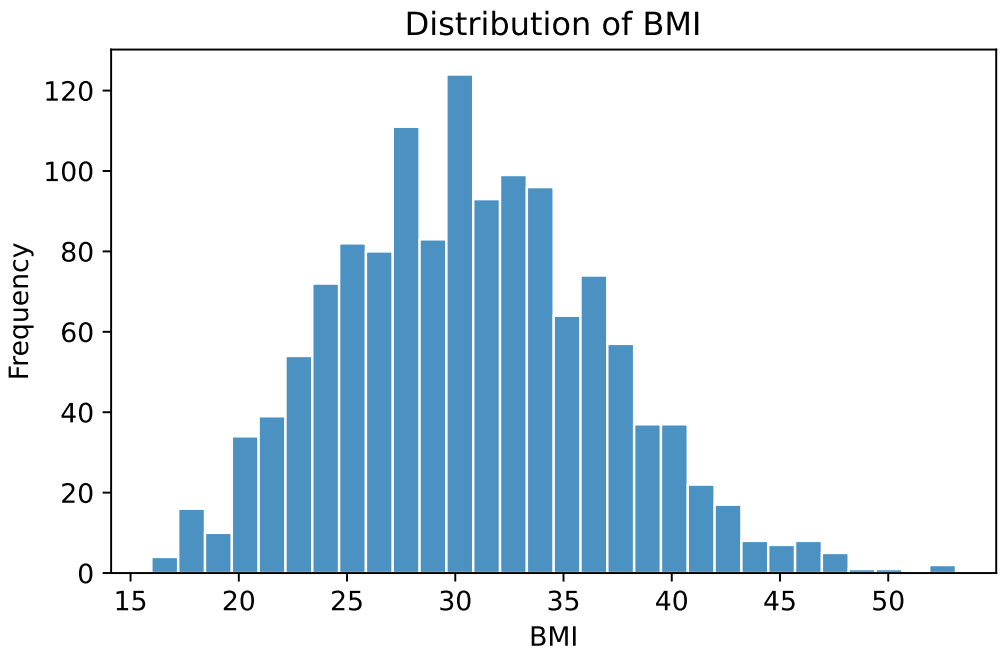
We visualise the distributions of the three numerical features—age, BMI, and number of children—to understand their spread and identify potential outliers.

```python
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(df['age'], bins=30, edgecolor='white', alpha=0.8)
ax.set_xlabel('Age')
ax.set_ylabel('Frequency')
ax.set_title('Distribution of Age')
plt.tight_layout()
fig.savefig('figures/fig_02a_distribution_age.pdf', bbox_inches='tight')
plt.show()
```



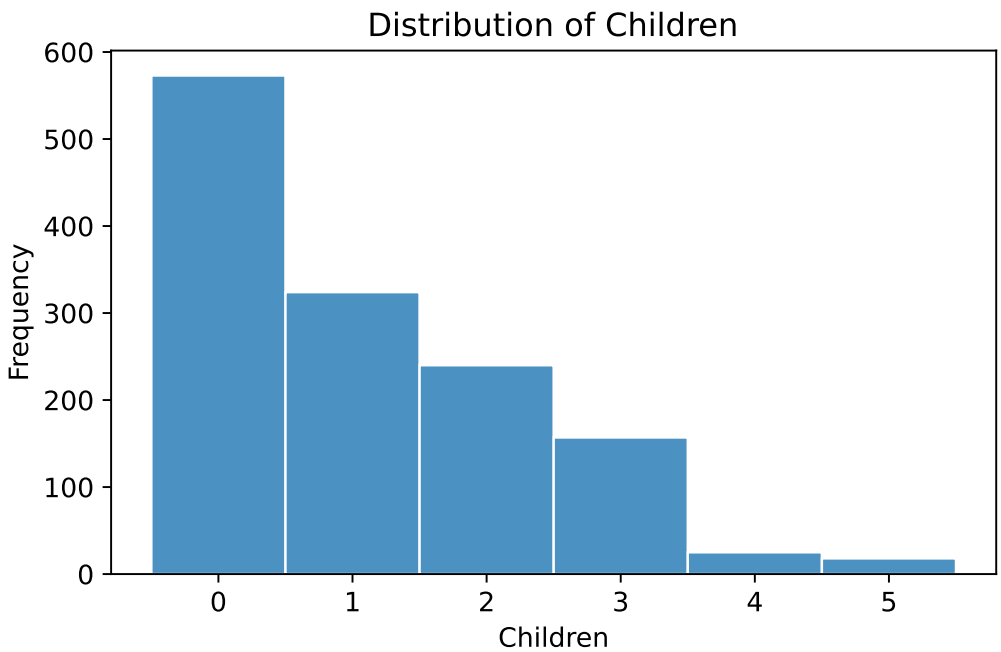
The age distribution spans 18 to 64 years with a mean of approximately 39 years. The relatively uniform spread ensures that the models observe a balanced range of ages during training.

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(df['bmi'], bins=30, edgecolor='white', alpha=0.8)
ax.set_xlabel('BMI')
ax.set_ylabel('Frequency')
ax.set_title('Distribution of BMI')
plt.tight_layout()
fig.savefig('figures/fig_02b_distribution_bmi.pdf', bbox_inches='tight')
plt.show()
```



BMI follows a roughly normal distribution with mean 30.70 and standard deviation 6.13. The centre of the distribution lies in the "obese" range (BMI ≥ 30), reflecting population-level trends.

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(df['children'], bins=range(int(df['children'].max()) + 2),
        edgecolor='white', alpha=0.8, align='left')
ax.set_xlabel('Children')
ax.set_ylabel('Frequency')
ax.set_title('Distribution of Children')
plt.tight_layout()
fig.savefig('figures/fig_02c_distribution_children.pdf', bbox_inches='tight')
plt.show()
```

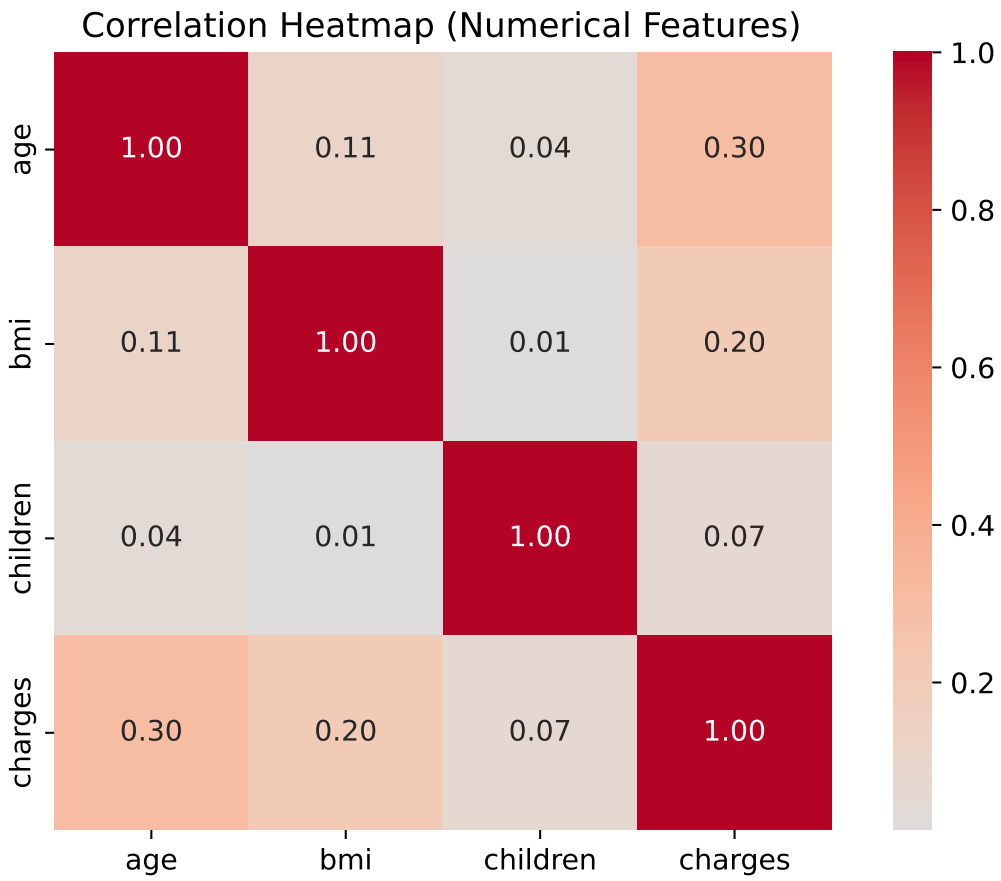


The number of children is a discrete variable ranging from 0 to 5. Most policyholders have zero or one dependant, and the frequency decreases monotonically as the number of children increases.

3.3 Correlation heatmap

We compute the pairwise Pearson correlation coefficients among the numerical features and the target variable to identify linear relationships.

```
fig, ax = plt.subplots(figsize=(7, 5))
corr = df[['age', 'bmi', 'children', 'charges']].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, ax=ax)
ax.set_title('Correlation Heatmap (Numerical Features)')
plt.tight_layout()
fig.savefig('figures/fig_03_correlation_heatmap.pdf', bbox_inches='tight')
plt.show()
```

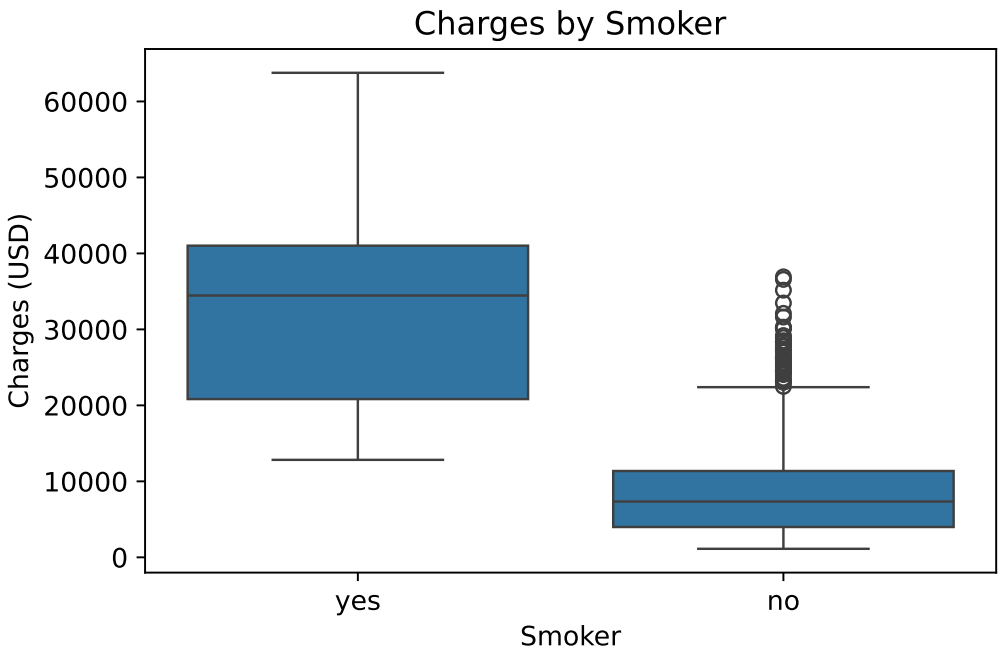


The heatmap reveals that `age` has the strongest linear correlation with `charges` among the numerical features. The correlation between `bmi` and `charges` is weaker in the marginal sense, but as we show later, this relationship strengthens dramatically when conditioned on smoking status. The `children` variable shows a near-zero correlation with charges.

3.4 Charges by categorical features

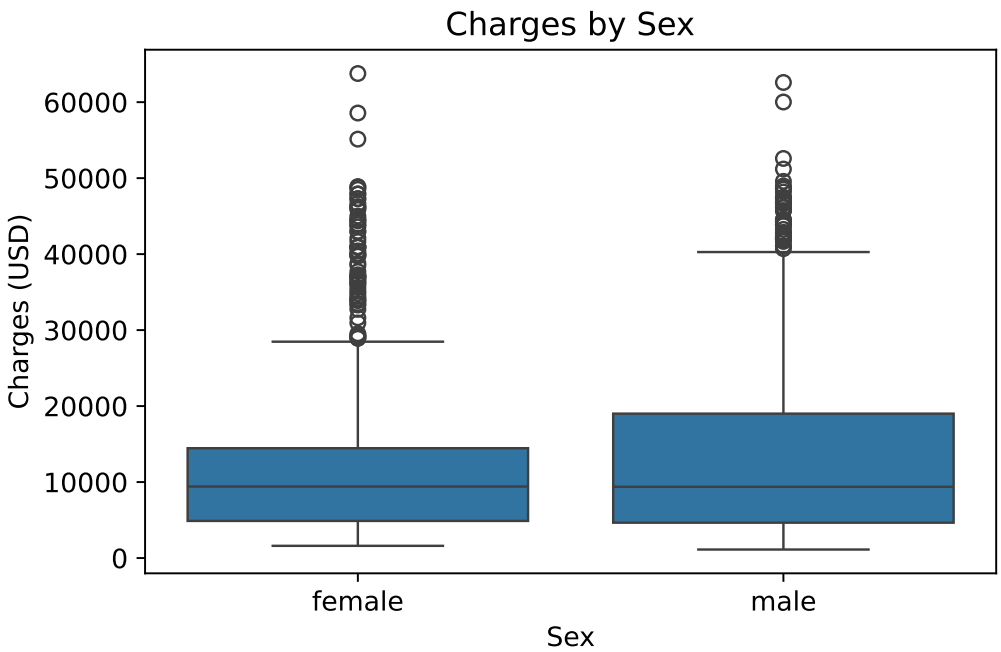
We use box plots to examine how the categorical features—smoking status, sex, and region—affect the distribution of charges.

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.boxplot(data=df, x='smoker', y='charges', ax=ax)
ax.set_title('Charges by Smoker')
ax.set_xlabel('Smoker')
ax.set_ylabel('Charges (USD)')
plt.tight_layout()
fig.savefig('figures/fig_04a_boxplot_smoker.pdf', bbox_inches='tight')
plt.show()
```



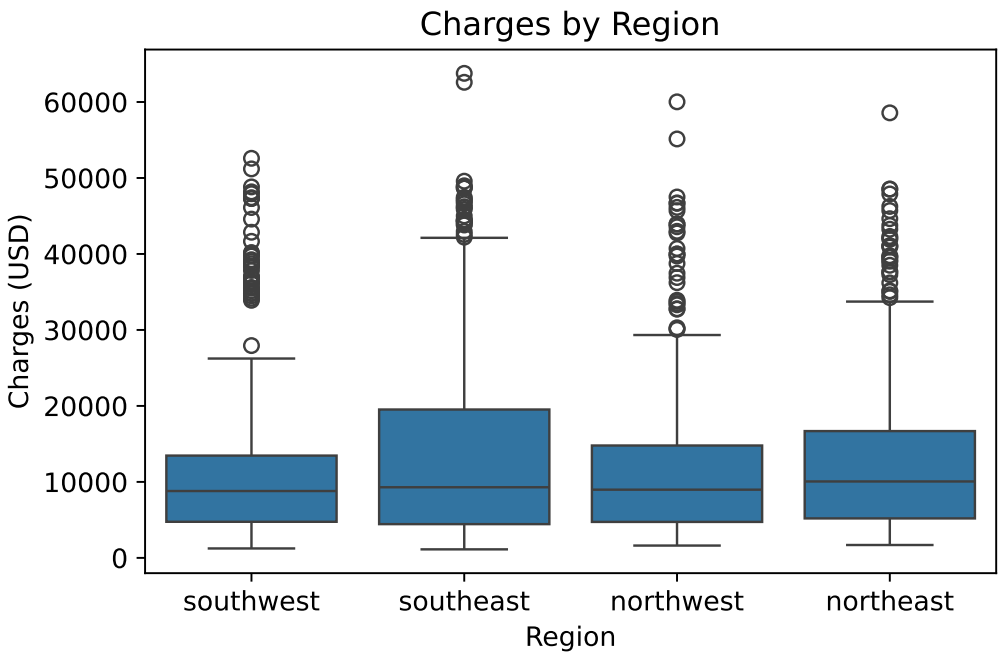
Smoking status produces the most striking separation in charges. The median charge for smokers is approximately three times higher than for non-smokers, and the interquartile range is also much wider, indicating greater variability in costs among smokers.

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.boxplot(data=df, x='sex', y='charges', ax=ax)
ax.set_title('Charges by Sex')
ax.set_xlabel('Sex')
ax.set_ylabel('Charges (USD)')
plt.tight_layout()
fig.savefig('figures/fig_04b_boxplot_sex.pdf', bbox_inches='tight')
plt.show()
```



The distributions of charges for female and male policyholders are nearly identical, with similar medians and interquartile ranges. Sex alone is a weak predictor of insurance costs.

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.boxplot(data=df, x='region', y='charges', ax=ax)
ax.set_title('Charges by Region')
ax.set_xlabel('Region')
ax.set_ylabel('Charges (USD)')
plt.tight_layout()
fig.savefig('figures/fig_04c_boxplot_region.pdf', bbox_inches='tight')
plt.show()
```

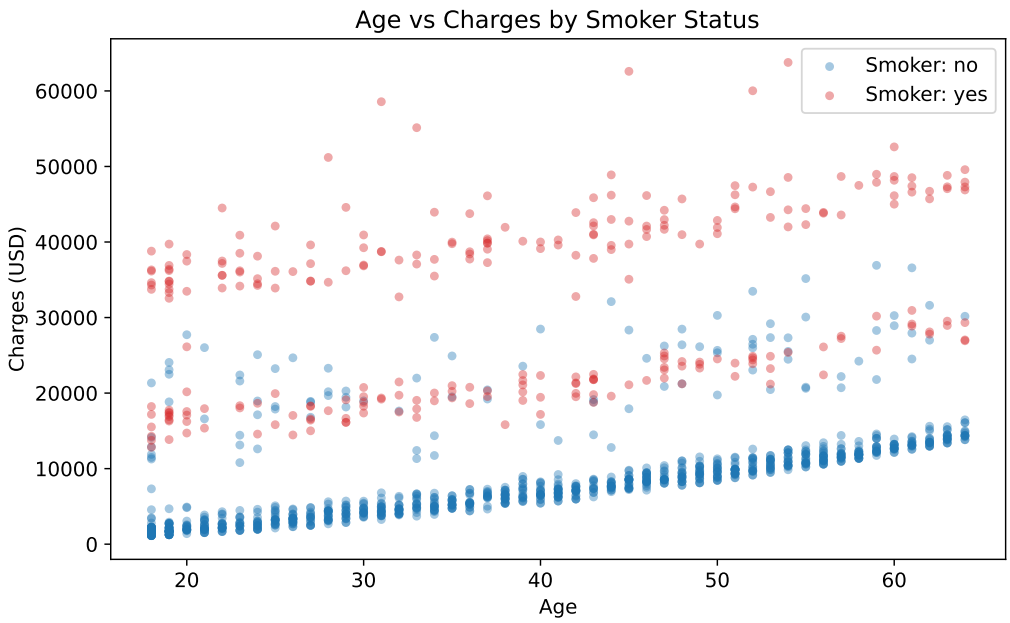


All four regions exhibit similar median charges and spread. While the southeast region shows slightly more outliers at the high end, the overall distributions are comparable. Region is therefore unlikely to be a dominant predictor on its own.

3.5 Age vs charges by smoker status

We investigate the relationship between age and charges, stratified by smoking status, to determine whether age affects smokers and non-smokers differently.

```
fig, ax = plt.subplots(figsize=(8, 5))
colors = {'yes': 'tab:red', 'no': 'tab:blue'}
for status, grp in df.groupby('smoker'):
    ax.scatter(grp['age'], grp['charges'], alpha=0.4, s=15,
               color=colors[status], label=f'Smoker: {status}')
ax.set_xlabel('Age')
ax.set_ylabel('Charges (USD)')
ax.set_title('Age vs Charges by Smoker Status')
ax.legend()
plt.tight_layout()
fig.savefig('figures/fig_05_age_vs_charges.pdf', bbox_inches='tight')
plt.show()
```

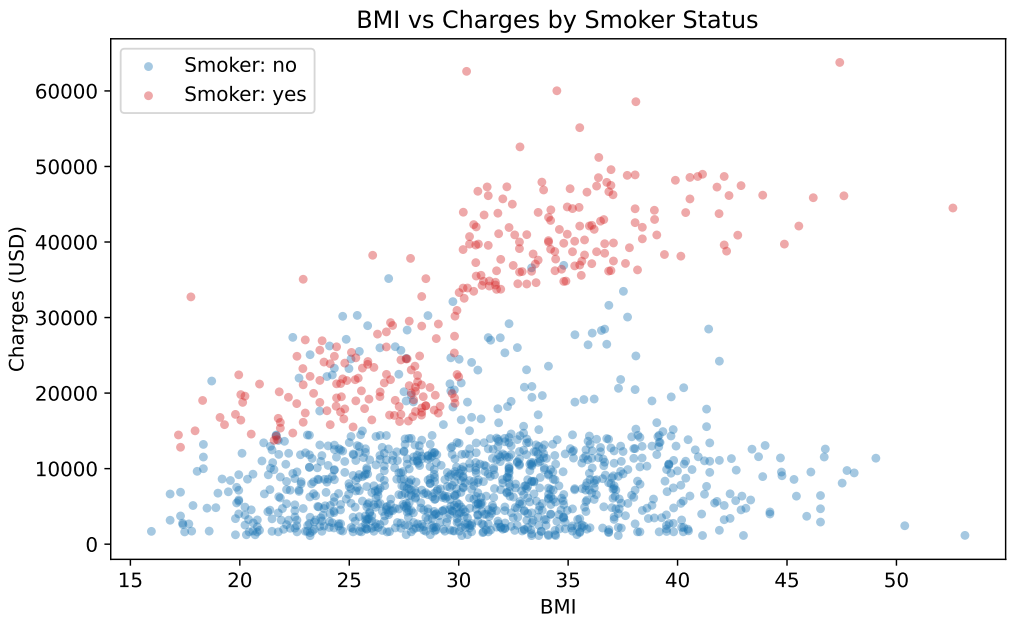


The scatter plot reveals two approximately linear bands. Non-smokers form a lower band where charges increase gradually with age. Smokers form a higher band with a steeper slope. This visual separation confirms that smoking status interacts with age to determine charges.

3.6 BMI vs charges by smoker status

We examine the relationship between BMI and charges, again stratified by smoking status, to identify potential interaction effects that motivate feature engineering.

```
fig, ax = plt.subplots(figsize=(8, 5))
for status, grp in df.groupby('smoker'):
    ax.scatter(grp['bmi'], grp['charges'], alpha=0.4, s=15,
               color=colors[status], label=f'Smoker: {status}')
ax.set_xlabel('BMI')
ax.set_ylabel('Charges (USD)')
ax.set_title('BMI vs Charges by Smoker Status')
ax.legend()
plt.tight_layout()
fig.savefig('figures/fig_06_bmi_vs_charges.pdf', bbox_inches='tight')
plt.show()
```



Among non-smokers, BMI shows little relationship with charges—the blue points form a roughly horizontal band. Among smokers, however, higher BMI drives charges up steeply, creating two distinct clusters. This observation motivates engineering a smoker–BMI interaction feature in Section 4.

4 Feature Engineering and Data Splitting

Based on the EDA findings, we create an interaction term between smoking status and BMI, apply one-hot encoding to the categorical features, and split the data into training and test sets.

4.1 Interaction term and encoding

The BMI vs charges scatter plot revealed that BMI has a strong positive effect on charges only for smokers. To capture this nonlinear relationship within a linear framework, we create an explicit interaction feature that equals the product of a binary smoking indicator and BMI.

```
# Feature engineering: smoker-BMI interaction
df['smoker_bmi'] = (df['smoker'] == 'yes').astype(int) * df['bmi']

# One-hot encoding
df_encoded = pd.get_dummies(df, columns=['sex', 'smoker', 'region'],
                            drop_first=True, dtype=int)

# Feature / target split
X = df_encoded.drop('charges', axis=1).astype(float)
y = df_encoded['charges']

print(f"Features ({len(X.columns)}): {list(X.columns)}")
```

The interaction feature `smoker_bmi` equals the product of a binary smoking indicator and BMI. For non-smokers it is zero; for smokers it equals their BMI value.

After one-hot encoding with `drop_first=True`, the encoded DataFrame `df_encoded` contains the original numerical columns plus binary indicators. The feature matrix $\mathbf{X}$ has shape $(1337, 9)$ with the following nine columns:

(1) `age` — policyholder age.

(2) `bmi` — body mass index.

(3) `children` — number of dependants.

(4) `smoker_bmi` — smoker–BMI interaction term.

(5) `sex_male` — 1 if male, 0 if female.

(6) `smoker_yes` — 1 if smoker, 0 if non-smoker.

(7) `region_northwest` — 1 if northwest region.

(8) `region_southeast` — 1 if southeast region.

(9) `region_southwest` — 1 if southwest region.

The target vector $\mathbf{y}$ contains $1,337$ charge values in USD.

4.2 Train/test split

We partition the data into training and test sets to enable unbiased evaluation of model performance on unseen observations.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Training set: {X_train.shape[0]} samples")
print(f"Test set:     {X_test.shape[0]} samples")
```

We split the data with an 80/20 ratio and a fixed random state for reproducibility. The training set `X_train` has shape $(1069, 9)$ containing $1,069$ samples, and the test set `X_test` has shape $(268, 9)$ containing 268 samples. The corresponding target vectors `y_train` and `y_test` have lengths $1,069$ and 268, respectively.

4.3 Feature scaling

We standardise all features to zero mean and unit variance so that distance-based and regularised models treat each feature on an equal footing.

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

We fit a `StandardScaler` on the training set only, then transform both the training and test sets. The scaler standardises each feature to zero mean and unit variance. The learned statistics are as follows:

Feature	Mean	Scale
age	39.20	13.99
bmi	30.54	6.05
children	1.08	1.19
smoker_bmi	6.06	12.43
sex_male	0.51	0.50
smoker_yes	0.20	0.40
region_northwest	0.25	0.43
region_southeast	0.27	0.44
region_southwest	0.25	0.43

Fitting on the training set only prevents information leakage from the test set into the preprocessing step.

5 Model Training

We train seven regression models, each with hyperparameter tuning via `GridSearchCV` using five-fold cross-validation and R^2 as the scoring metric. All models receive the scaled features. Tree-based models do not require scaling, but applying it uniformly does not harm their performance.

We define a helper function `tune_model` that centralises the grid-search workflow.

```
best_models = {}
cv_results = []

def tune_model(name, estimator, param_grid):
    """Run GridSearchCV on the training set and store the best estimator."""
    gs = GridSearchCV(estimator, param_grid, cv=5, scoring='r2', n_jobs=-1)
    gs.fit(X_train_sc, y_train)
    best_models[name] = gs.best_estimator_
    cv_results.append({
        'Model': name,
        'Best CV R2': gs.best_score_,
        'Best Params': gs.best_params_,
    })
    print(f"{name}:  CV R2 = {gs.best_score_:.4f},  {gs.best_params_}")
```

The function accepts a model name, an unfitted estimator, and a parameter grid. It fits a `GridSearchCV` object on the scaled training data, stores the `best_estimator_` in the dictionary `best_models`, and appends the `best_score_` and `best_params_` to the list `cv_results`.

5.1 Linear regression

We begin with ordinary least squares as a baseline model. The objective is to minimise the residual sum of squares:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \left(y_i - \mathbf{x}_i^\top \boldsymbol{\beta}\right)^2,$$

with prediction $\hat{y} = \mathbf{x}^\top \hat{\boldsymbol{\beta}}$. Linear regression has no tunable hyperparameters, so the grid search reduces to a single fit.

```
tune_model('Linear Regression', LinearRegression(), {})
```

Parameter	Values
(none)	—

Linear regression has no tunable hyperparameters. The best cross-validation R^2 is 0.8218. The `best_estimator_` is a standard `LinearRegression` instance.

5.2 Ridge regression

We add ℓ_2 regularisation to the linear model. The ridge objective augments the ordinary least squares loss with a squared-norm penalty:

$$\hat{\boldsymbol{\beta}}_{\text{ridge}} = \arg\min_{\boldsymbol{\beta}} \left\{ \sum_{i=1}^{n} \left(y_i - \mathbf{x}_i^\top \boldsymbol{\beta}\right)^2 + \alpha \|\boldsymbol{\beta}\|_2^2 \right\}.$$

We search over a range of penalty strengths to determine whether shrinkage improves generalisation.

```
tune_model('Ridge Regression', Ridge(), {
    'alpha': [0.1, 1.0, 10.0, 100.0]
})
```

Parameter	Values
`alpha`	0.1, 1.0, 10.0, 100.0

The best regularisation strength is $\alpha = 0.1$, yielding a cross-validation R^2 of 0.8218. The performance matches ordinary linear regression, indicating that the features are well-conditioned and ℓ_2 regularisation provides no additional benefit at this scale.

5.3 Lasso regression

We apply ℓ_1 regularisation, which can drive some coefficients exactly to zero, effectively performing feature selection during training. The lasso objective is:

$$\hat{\boldsymbol{\beta}}_{\text{lasso}} = \arg\min_{\boldsymbol{\beta}} \left\{ \sum_{i=1}^{n} \left(y_i - \mathbf{x}_i^\top \boldsymbol{\beta}\right)^2 + \alpha \|\boldsymbol{\beta}\|_1 \right\}.$$

```
tune_model('Lasso Regression', Lasso(), {
    'alpha': [0.1, 1.0, 10.0, 100.0]
})
```

Parameter	Values
`alpha`	0.1, 1.0, 10.0, 100.0

The best lasso parameter is $\alpha = 1.0$ with a cross-validation R^2 of 0.8218. Performance is effectively identical to ordinary least squares, confirming that feature selection via ℓ_1 penalisation does not improve the fit in this setting.

5.4 Support vector regression

We train a support vector regressor with an ε-insensitive loss function. In a reproducing kernel Hilbert space $\mathcal{H}_K$ induced by a kernel function K, the objective is:

$$\min_{f \in \mathcal{H}_K} \frac{1}{2} \|f\|_{\mathcal{H}_K}^2 + C \sum_{i=1}^{n} \max\left(0, , |y_i - f(\mathbf{x}_i)| - \varepsilon\right),$$

where C controls regularisation and ε sets the width of the insensitive tube. The kernel K maps the inputs into a higher-dimensional feature space; we search over linear, polynomial, RBF, and sigmoid kernels alongside C and ε.

```
tune_model('SVR', SVR(), {
    'C': [1000, 5000, 10000],
    'epsilon': [0.1, 1.0, 10.0],
    'kernel': ['linear', 'poly', 'rbf', 'sigmoid']
})
```

Parameter	Values
`C`	1,000, 5,000, 10,000
`epsilon`	0.1, 1.0, 10.0
`kernel`	linear, poly, rbf, sigmoid

The best configuration is $C = 10,000$, $\varepsilon = 10.0$, and the RBF kernel, achieving a cross-validation R^2 of 0.8127. The large ε value means the model tolerates substantial deviations before incurring a penalty, reflecting the high variability in charges.

5.5 Decision tree

We train a decision tree regressor $T(\mathbf{x}; , d_{\max}, , n_{\min})$, where $d_{\max}$ is the maximum depth and $n_{\min}$ is the minimum number of samples required to split an internal node. At each node of depth less than $d_{\max}$ with at least $n_{\min}$ samples, the tree selects the feature j and threshold t that minimise the weighted mean squared error of the resulting partition:

$$\min_{j,,t} \left[\frac{n_L}{n}, \mathrm{MSE}(R_L) + \frac{n_R}{n}, \mathrm{MSE}(R_R) \right], \quad \mathrm{MSE}(R) = \frac{1}{|R|} \sum_{i \in R} (y_i - \bar{y}_R)^2.$$

We tune both $d_{\max}$ and $n_{\min}$ to control model complexity.

```
tune_model('Decision Tree', DecisionTreeRegressor(random_state=42), {
    'max_depth': [3, 5, 7, 10],
    'min_samples_split': [2, 5, 10]
})
```

Parameter	Values
`max_depth`	3, 5, 7, 10
`min_samples_split`	2, 5, 10

The best tree has `max_depth` $= 5$ and `min_samples_split` $= 2$, achieving a cross-validation R^2 of 0.8282. Limiting the depth to 5 prevents overfitting while still capturing key interactions such as the smoker–BMI split.

5.6 Random forest

We train a random forest [1], an ensemble of decorrelated decision trees. The ensemble prediction averages B independently trained trees, each fitted on a bootstrap sample:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x}),$$

where T_b denotes the b-th decision tree. Each tree is constrained by a maximum depth $d_{\max}$ and a minimum number of samples n_{leaf} at each leaf node, and each split considers a random subset of features to decorrelate the trees. We tune the number of estimators B, $d_{\max}$, and n_{leaf}.

```
tune_model('Random Forest', RandomForestRegressor(random_state=42), {
    'n_estimators': [100, 200],
    'max_depth': [5, 10],
    'min_samples_leaf': [1, 4]
})
```

Parameter	Values
`n_estimators`	100, 200
`max_depth`	5, 10
`min_samples_leaf`	1, 4

The best random forest uses 200 trees with `max_depth` = 5 and `min_samples_leaf` = 4, achieving the highest cross-validation R^2 of 0.8444. The ensemble of shallow trees reduces variance compared to a single decision tree.

5.7 Gradient boosting

We train a gradient boosting regressor, which builds M trees sequentially to correct the residual errors of previous trees. At stage m the ensemble is:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta, h_m(\mathbf{x}), \quad m = 1, \ldots, M,$$

where h_m is a decision tree of maximum depth $d_{\max}$ fitted to the negative gradient of the loss, η is the learning rate, and M is the total number of boosting stages. We tune η, M, and $d_{\max}$.

```
tune_model('Gradient Boosting', GradientBoostingRegressor(random_state=42), {
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5]
})
```

Parameter	Values
`n_estimators`	100, 200
`learning_rate`	0.05, 0.1
`max_depth`	3, 5

The best gradient boosting configuration uses 100 trees with a learning rate of 0.05 and `max_depth` = 3, yielding a cross-validation R^2 of 0.8425. The smaller learning rate combined with fewer trees suggests that the model benefits from a conservative, incremental approach to fitting residuals.

6 Model Evaluation

We evaluate each tuned model on the held-out test set using two metrics: **mean absolute error** (MAE) and the **coefficient of determination** (R^2). For each model, we compute predictions `y_pred` on the scaled test features `X_test_sc`.

6.1 Linear regression

We apply the tuned linear regression model to the test set and compute the evaluation metrics.

```
y_pred_lr = best_models['Linear Regression'].predict(X_test_sc)
print("Linear Regression")
print(f"  Test MAE: ${mean_absolute_error(y_test, y_pred_lr):,.2f}")
print(f"  Test R2:  {r2_score(y_test, y_pred_lr):.4f}")
```

The prediction vector `y_pred_lr` contains 268 predicted charge values. Linear regression achieves a test MAE of \$2,828.97 and a test R^2 of 0.8862, explaining approximately 88.6% of the variance in charges on unseen data.

6.2 Ridge regression

We evaluate the tuned ridge regression model on the test set.

```
y_pred_ridge = best_models['Ridge Regression'].predict(X_test_sc)
print("Ridge Regression")
print(f"  Test MAE: ${mean_absolute_error(y_test, y_pred_ridge):,.2f}")
print(f"  Test R2:  {r2_score(y_test, y_pred_ridge):.4f}")
```

Ridge regression achieves a test MAE of \$2,829.04 and a test R^2 of 0.8863, essentially identical to ordinary linear regression.

6.3 Lasso regression

We evaluate the tuned lasso regression model on the test set.

```
y_pred_lasso = best_models['Lasso Regression'].predict(X_test_sc)
print("Lasso Regression")
print(f"  Test MAE: ${mean_absolute_error(y_test, y_pred_lasso):,.2f}")
print(f"  Test R2:  {r2_score(y_test, y_pred_lasso):.4f}")
```

Lasso regression achieves a test MAE of \$2,828.98 and a test R^2 of 0.8862, again matching the linear baseline.

6.4 Support vector regression

We evaluate the tuned SVR model on the test set.

```
y_pred_svr = best_models['SVR'].predict(X_test_sc)
print("SVR")
print(f"  Test MAE: ${mean_absolute_error(y_test, y_pred_svr):,.2f}")
print(f"  Test R2:  {r2_score(y_test, y_pred_svr):.4f}")
```

SVR achieves the lowest MAE of \$1,875.34 but a slightly lower R^2 of 0.8790. The low MAE reflects the ε-insensitive loss function, which focuses on reducing typical errors rather than large outlier errors that disproportionately affect R^2.

6.5 Decision tree

We evaluate the tuned decision tree on the test set.

```
y_pred_dt = best_models['Decision Tree'].predict(X_test_sc)
print("Decision Tree")
print(f"  Test MAE: ${mean_absolute_error(y_test, y_pred_dt):,.2f}")
print(f"  Test R2:  {r2_score(y_test, y_pred_dt):.4f}")
```

The decision tree achieves a test MAE of \$2,681.69 and a test R^2 of 0.8915, outperforming all three linear models on R^2.

6.6 Random forest

We evaluate the tuned random forest on the test set.

```
y_pred_rf = best_models['Random Forest'].predict(X_test_sc)
print("Random Forest")
print(f"  Test MAE: ${mean_absolute_error(y_test, y_pred_rf):,.2f}")
print(f"  Test R2:  {r2_score(y_test, y_pred_rf):.4f}")
```

The random forest achieves a test MAE of \$2,413.46 and a test R^2 of 0.9012—the highest R^2 among all seven models.

6.7 Gradient boosting

We evaluate the tuned gradient boosting model on the test set.

```
y_pred_gb = best_models['Gradient Boosting'].predict(X_test_sc)
print("Gradient Boosting")
print(f"  Test MAE: ${mean_absolute_error(y_test, y_pred_gb):,.2f}")
print(f"  Test R2:  {r2_score(y_test, y_pred_gb):.4f}")
```

Gradient boosting achieves a test MAE of \$2,463.99 and a test R^2 of 0.9011, nearly matching the random forest.

6.8 Summary of test results

Table 1 consolidates the test-set performance of all seven models.

Table 1: Test-set performance of all seven models.

Model	Test MAE (USD)	Test R^2
Random Forest	2,413.46	0.9012
Gradient Boosting	2,463.99	0.9011
Decision Tree	2,681.69	0.8915
Ridge Regression	2,829.04	0.8863
Linear Regression	2,828.97	0.8862
Lasso Regression	2,828.98	0.8862
SVR	1,875.34	0.8790

7 Model Comparison

We consolidate cross-validation and test-set results into a single comparison table and visualise the differences.

```
test_preds = {
    'Linear Regression': y_pred_lr,
    'Ridge Regression':  y_pred_ridge,
    'Lasso Regression':  y_pred_lasso,
    'SVR':               y_pred_svr,
    'Decision Tree':     y_pred_dt,
    'Random Forest':     y_pred_rf,
    'Gradient Boosting': y_pred_gb,
}

rows = []
for rec in cv_results:
    name = rec['Model']
    yp = test_preds[name]
    rows.append({
        'Model':    name,
        'CV R2':    rec['Best CV R2'],
        'Test MAE': mean_absolute_error(y_test, yp),
        'Test R2':  r2_score(y_test, yp),
    })

comparison_df = pd.DataFrame(rows).sort_values('Test R2', ascending=False)
```

Table 2: Full model comparison: cross-validation R^2, test MAE, and test R^2.

Model	CV R^2	Test MAE (USD)	Test R^2
Random Forest	0.8444	2,413.46	0.9012
Gradient Boosting	0.8425	2,463.99	0.9011
Decision Tree	0.8282	2,681.69	0.8915
Ridge Regression	0.8218	2,829.04	0.8863
Linear Regression	0.8218	2,828.97	0.8862
Lasso Regression	0.8218	2,828.98	0.8862
SVR	0.8127	1,875.34	0.8790

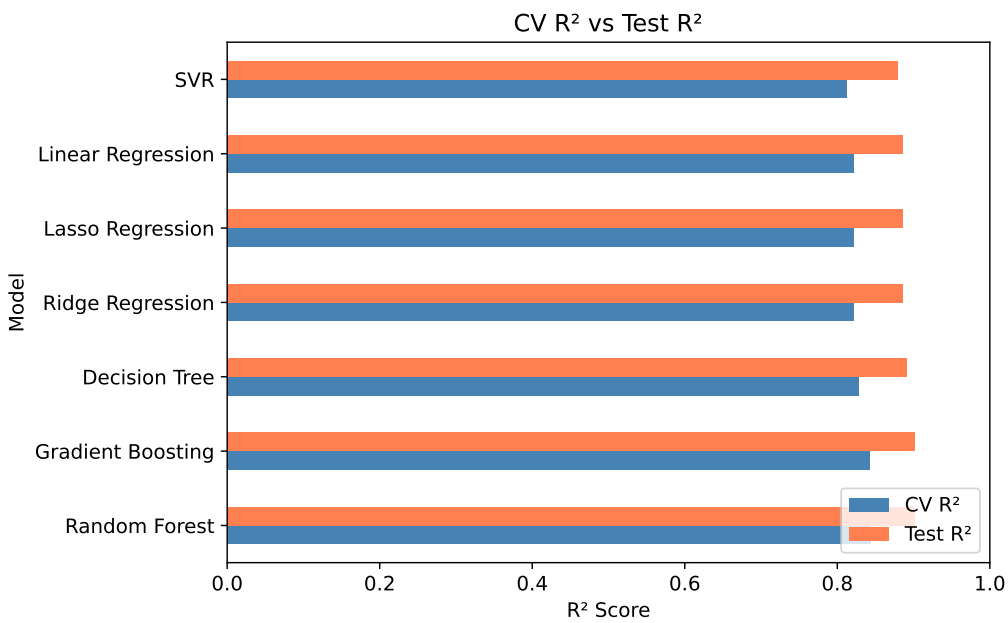
The DataFrame `comparison_df` contains four columns—`Model`, `CV R²`, `Test MAE`, and `Test R²`—sorted by descending test R^2. Table 2 presents the full comparison.

The random forest ranks first by test R^2 (0.9012), followed closely by gradient boosting (0.9011). The three linear models cluster together at approximately 0.886. SVR has the lowest R^2 (0.8790) but the best MAE ($1,875.34), illustrating that different metrics can lead to different model rankings.

7.1 CV R^2 vs test R^2

We visualise the cross-validation and test R^2 scores side by side to assess whether any model overfits or underfits relative to its training performance.

```
fig, ax = plt.subplots(figsize=(8, 5))
plot_df = comparison_df.set_index('Model')[['CV R2', 'Test R2']]
plot_df.plot(kind='barh', ax=ax, color=['steelblue', 'coral'])
ax.set_xlabel('R2 Score')
ax.set_title('CV R2 vs Test R2')
ax.set_xlim(0, 1)
ax.legend(loc='lower right')
plt.tight_layout()
fig.savefig('figures/fig_07a_model_r2_comparison.pdf', bbox_inches='tight')
plt.show()
```

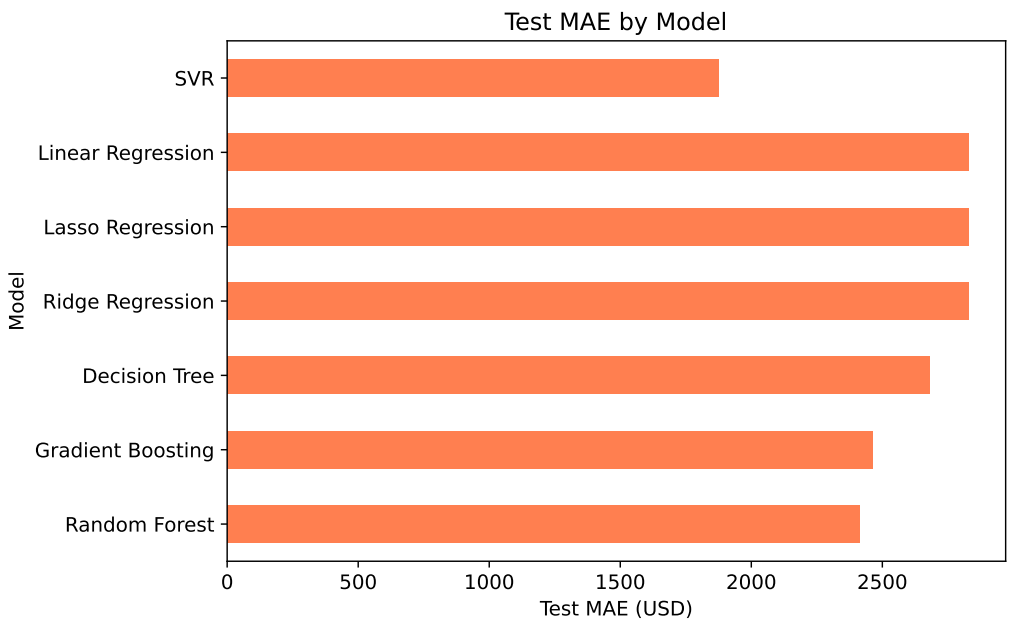


Every model achieves a higher R^2 on the test set than in cross-validation. This suggests that the particular test split happens to be slightly easier to predict than the average cross-validation fold—a natural consequence of random splitting with a relatively small dataset.

7.2 Test MAE comparison

We compare the test MAE across all models to identify which model produces the smallest typical prediction error.

```
fig, ax = plt.subplots(figsize=(8, 5))
comparison_df.set_index('Model')['Test MAE'].plot(
    kind='barh', ax=ax, color='coral'
)
ax.set_xlabel('Test MAE (USD)')
ax.set_title('Test MAE by Model')
plt.tight_layout()
fig.savefig('figures/fig_07b_model_mae_comparison.pdf', bbox_inches='tight')
plt.show()
```



SVR's low MAE indicates that its typical prediction error is smaller than that of the tree-based models, even though its overall variance explained (R^2) is lower. This trade-off occurs because the ε-insensitive loss is robust to outliers, whereas R^2 penalises large squared errors heavily.

7.3 Best model selection

We select the best model based on the highest test R^2, which measures the proportion of variance in the target variable explained by the model on unseen data.

```
best_row = comparison_df.iloc[0]
best_name = best_row['Model']
best_model = best_models[best_name]

print(f"Best model: {best_name}")
print(f"  CV R2:    {best_row['CV R2']:.4f}")
print(f"  Test MAE: ${best_row['Test MAE']:,.2f}")
print(f"  Test R2:  {best_row['Test R2']:.4f}")
```

We select the **random forest** as the best model based on the highest test R^2 of 0.9012. Its cross-validation R^2 is 0.8444 and its test MAE is \$2,413.46. The model explains over 90% of the variance in unseen insurance charges.

8 Model Deployment

We serialise the best model, the scaler, and the feature list into a single pickle file for future inference.

8.1 Saving the artifact

We bundle the trained model, the fitted scaler, and the ordered feature list into a single dictionary and serialise it using Python's `pickle` module.

```
artifact = {
    'model': best_model,
    'model_name': best_name,
    'scaler': scaler,
    'features': list(X.columns),
}

with open('medical_insurance_model.pkl', 'wb') as f:
    pickle.dump(artifact, f)

print(f"Saved '{best_name}' to medical_insurance_model.pkl")
```

The dictionary `artifact` contains four keys:

- `model` — the fitted `RandomForestRegressor` instance with 200 trees, `max_depth` = 5, and `min_samples_leaf` = 4.
- `model_name` — the string `"Random Forest"`.
- `scaler` — the fitted `StandardScaler` that learned the training-set mean and standard deviation for each feature.
- `features` — the ordered list of nine feature names.

Bundling these together ensures that any future user can reproduce the exact preprocessing and prediction pipeline without access to the training data.

8.2 Loading and inspecting the artifact

We reload the serialised artifact from disk to verify that the model, scaler, and feature list were persisted correctly.

```
with open('medical_insurance_model.pkl', 'rb') as f:
    artifact = pickle.load(f)

print(f"Keys:       {list(artifact.keys())}")
print(f"Model name: {artifact['model_name']}")
print(f"Model type: {type(artifact['model']).__name__}")
```

```
Keys:       ['model', 'model_name', 'scaler', 'features']
Model name: Random Forest
Model type: RandomForestRegressor
```

```python
print("Features:", artifact['features'])
print(f"\nScaler:   {type(artifact['scaler']).__name__}")
print(f"  mean:   {np.round(artifact['scaler'].mean_, 2)}")
print(f"  scale:  {np.round(artifact['scaler'].scale_, 2)}")
```

```
Features: ['age', 'bmi', 'children', 'smoker_bmi', 'sex_male',
           'smoker_yes', 'region_northwest', 'region_southeast',
           'region_southwest']

Scaler:   StandardScaler
  mean:   [39.2  30.54  1.08  6.06  0.51  0.2   0.25  0.27  0.25]
  scale:  [13.99  6.05  1.19 12.43  0.5   0.4   0.43  0.44  0.43]
```

The scaler's learned means and standard deviations confirm reasonable training-set statistics. For example, the mean age is 39.2 years and the mean BMI is 30.54. The `smoker_bmi` feature has a mean of 6.06, reflecting the fact that only about 20% of policyholders are smokers (mean of `smoker_yes` = 0.2).

8.3 Prediction on a new observation

We demonstrate the end-to-end inference pipeline by constructing a single synthetic observation, scaling it with the stored scaler, and generating a predicted charge.

```python
# Construct a single unseen observation
new_obs = pd.DataFrame([{
    'age': 45, 'bmi': 28.5, 'children': 2,
    'smoker_bmi': 1 * 28.5,
    'sex_male': 1,
    'smoker_yes': 1,
    'region_northwest': 0,
    'region_southeast': 1,
    'region_southwest': 0,
}])[artifact['features']]

print("Input (raw):")
print(new_obs.to_string(index=False))

new_obs_sc = artifact['scaler'].transform(new_obs)
predicted = artifact['model'].predict(new_obs_sc)[0]

print(f"\nPredicted charge: ${predicted:,.2f}")
```

```
Input (raw):
 age  bmi  children  smoker_bmi  sex_male  smoker_yes  region_northwest
  45 28.5         2        28.5         1           1                 0

 region_southeast  region_southwest
                1                 0

Predicted charge: $26,325.57
```

The synthetic observation describes a 45-year-old male smoker with a BMI of 28.5, two children, living in the southeast region. Since the policyholder is a smoker, the `smoker_bmi` interaction term equals $1 \times 28.5 = 28.5$. After scaling the raw features with the stored scaler, the random forest predicts an annual charge of \$26,325.57. This high predicted cost is consistent with the strong influence of smoking status and the smoker–BMI interaction observed during exploratory data analysis.

9 Conclusion

We have completed a data science workflow for predicting medical insurance charges from demographic and health-related features.

Data cleaning. The original dataset contained 2,772 rows with 1,435 exact duplicates. After deduplication, 1,337 unique records remained with no missing values.

EDA findings. The target variable is right-skewed (skewness = 1.52), with a mean of \$13,279.12 and a median of \$9,386.16. Smoking status emerged as the dominant predictor. Among smokers, higher BMI drives charges up steeply, motivating the creation of a smoker–BMI interaction feature.

Model comparison. We trained and tuned seven regression models via five-fold cross-validation and evaluated each on a held-out test set. The random forest achieved the highest test R^2 of 0.9012 with a test MAE of \$2,413.46, followed closely by gradient boosting ($R^2 = 0.9011$). The three linear models (ordinary, ridge, and lasso) performed identically at approximately $R^2 = 0.886$, while SVR achieved the lowest MAE (\$1,875.34) but also the lowest R^2 (0.8790).

Practical implications. The strong influence of smoking on predicted charges aligns with actuarial knowledge. The interaction between smoking and BMI suggests that targeted health interventions for smokers with high BMI could have the largest impact on reducing insurance costs. The exported pickle artifact enables real-time charge estimation for new policyholders.

Limitations. The dataset is relatively small ($n = 1,337$ after deduplication), which limits the complexity of models we can reliably train. The large number of duplicates in the raw data raises questions about data collection methodology. Additionally, important predictors such as pre-existing conditions, medication usage, and lifestyle factors are absent.

Future work. Several directions could improve performance:

(i) Collecting a larger, deduplicated dataset with richer features such as pre-existing conditions and prescription history.

(ii) Exploring nonlinear transformations of the target variable (e.g., log-transform) to address skewness and potentially improve linear model performance.

(iii) Testing additional ensemble methods such as XGBoost or LightGBM with more extensive hyperparameter grids.

(iv) Incorporating domain knowledge through feature engineering, such as age-group binning or WHO BMI categories.

References

[1] L. Breiman. "Random forests". In: *Machine learning* 45.1 (2001), pp. 5–32.

[2] J. A. Cenita, P. R. Asuncion, and J. Victoriano. "Performance Evaluation of Regression Models in Predicting the Cost of Medical Insurance". In: *International Journal of Computing Sciences Research* 7 (2023), pp. 2052–2065. ISSN: 2546-115X. URL: https://stepacademic.net/ijcsr/article/view/417.

[3] M. Choi. *Medical Cost Personal Datasets*. 2018. URL: https://www.kaggle.com/datasets/mirichoi0218/insurance.

[4] C. R. Harris et al. "Array programming with NumPy". In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: https://doi.org/10.1038/s41586-020-2649-2.

[5] J. D. Hunter. "Matplotlib: A 2D graphics environment". In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: 10.1109/MCSE.2007.55.

[6] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[7] T. pandas development team. *pandas-dev/pandas: Pandas*. 2020. DOI: 10.5281/zenodo.3509134. URL: https://doi.org/10.5281/zenodo.3509134.

[8] G. Van Rossum and F. L. Drake. *Python library reference*. Centrum voor Wiskunde en Informatica Amsterdam, 1995.

[9] M. L. Waskom. "seaborn: statistical data visualization". In: *Journal of Open Source Software* 6.60 (2021), p. 3021. DOI: 10.21105/joss.03021. URL: https://doi.org/10.21105/joss.03021.